
Typed Planning and Deterministic Execution for Verifiable Public-Procurement Question Answering

Student Name*

Department and University

student@example.ac.uk

Abstract

Public-procurement data can support questions about buyers, suppliers, contract notices, and disclosure, but many useful questions require exact set construction, multi-hop joins, aggregation, and comparison rather than retrieval of a few relevant passages. We present a question-answering pipeline that maps natural-language questions to programs in a closed, typed algebra and delegates factual authority to deterministic execution over a provenance-preserving procurement knowledge graph. Training programs are generated from the data and retained only after type checking, non-degeneracy checks, and agreement between two execution implementations. For deployment, a structured understander is followed by either a deterministic intent compiler or, when the intent cannot be compiled safely, a typed graph planner; grounding, exhaustive execution, evidence checks, and bounded replanning then produce an answer or a controlled non-answer. On the frozen 922-item PACS compositional test, an 8B parameter model trained with typed programs obtains 74.51% strict execution accuracy on the canonical surface and 73.21% on an independently naturalised surface, compared with 36.01% for the same untuned base model and 48.81% for a cloud-model planner. On a separate balanced 260-item end-to-end snapshot, the full local runtime scores 86.15%, compared with 70.38% for its untuned counterpart. The results support typed execution as a practical way to improve compositional reliability, while audits also expose unresolved limitations in currency normalisation, entity aliases, benchmark isolation, and artifact portability.

1 Introduction

Public procurement records describe how public buyers solicit work, choose suppliers, and publish awards and contracts. The Open Contracting Data Standard (OCDS) gives these disclosures a common model covering parties, tenders, awards, contracts, and linked documents ([Open Contracting Partnership, 2026](#)). Data published this way can answer questions such as which suppliers serve several buyers, whether disclosure is complete for a class of notices, or how activity shifts across years. None of these is a document lookup. An answer may depend on the role an organisation plays in one record, on a set of organisations derived from that record, on a second retrieval bound by the derived set, and on a final count, comparison, or ranking.

Finding *relevant* evidence is not the same as recovering the *complete* denotation an aggregate needs. Let $R(q)$ be the full set of contract records satisfying question q , and let a top- k retriever return $\hat{R}(q) \subset R(q)$. Even when $\hat{R}(q)$ holds convincing examples, it does not follow that

$$|\hat{R}(q)| = |R(q)|, \quad \sum_{r \in \hat{R}(q)} v(r) = \sum_{r \in R(q)} v(r), \quad (1)$$

nor that a ranking computed from the retrieved subset shares the true maximum. Retrieval-centred KGQA systems find answer-bearing subgraphs well ([Sun et al., 2019](#); [Jiang et al., 2023b](#); [Luo et al., 2024](#)). Exact

*This draft was prepared with language-model assistance for literature search, organisation, and language editing. The author is responsible for checking every claim, experiment, citation, and final wording.

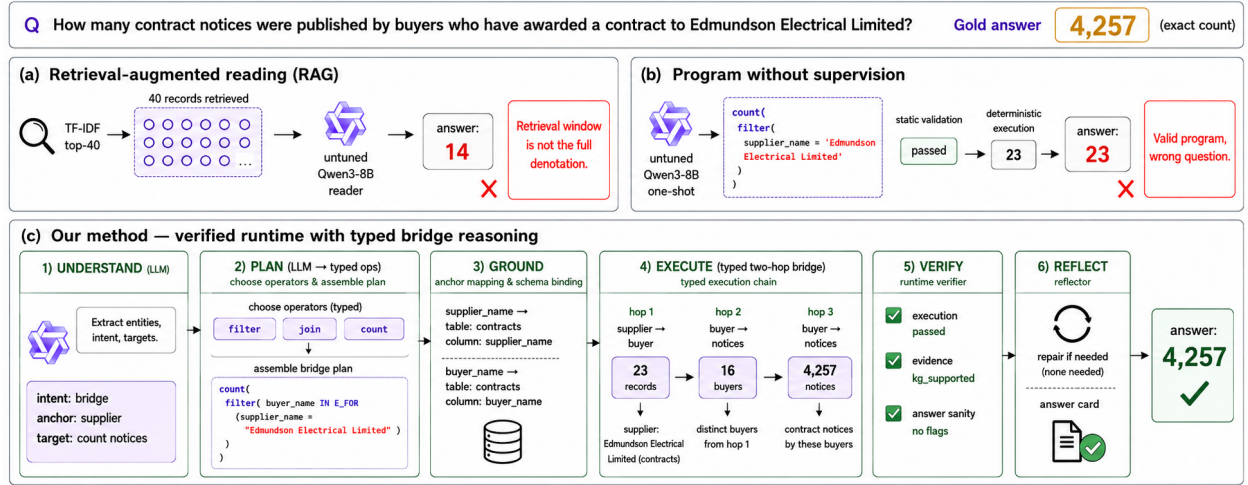


Figure 1: One bridge question over the frozen procurement graph. Retrieval-augmented reading retrieves 40 records, among them all 23 naming the supplier, and answers 14; an untuned model of the same size emits a program that type checks and executes cleanly but stops after the first hop and returns 23. Our runtime executes the two-hop bridge exhaustively—23 source records, 16 buyer values, 4,257 notices—and releases an answer only once execution, evidence, and answer-sanity checks pass. Every value comes from one saved run; its provenance and limits are given in Appendix E.

counts and set operations need something further: an explicit scope, typed intermediate values, and exhaustive execution inside a declared data universe.

Letting a language model write the answer directly fails differently. A model can produce a well-formed query that swaps buyer and supplier roles, projects a contract identifier where the question asked for an organisation name, or sums a non-additive monetary field. The answers stay fluent and look plausible. Prior work has responded by representing complex questions as executable programs (Cao et al., 2022; Wolfson et al., 2020), constraining formal generation (Scholak et al., 2021; Shu et al., 2022), and feeding execution or environment signals back into the parser (Chen et al., 2021; Huang et al., 2024). We take that starting point and push it into a division of labour: the model proposes a typed computation, and the data system decides every factual value.

We build this at two levels, and keep them separate in the evaluation as well as in the code. A compact model is trained to translate procurement questions into a closed algebra of record filters, distinct projections, semi- and anti-joins, set operations, grouping, aggregation, comparison, and ranking; each program it emits is type checked and executed over a frozen record universe, and an offline scorer compares the result with a fixed oracle. Around that model, a deployment runtime adds a structured understanding stage and a conditional planner. Intents that compile deterministically take a compiler path and the rest go to a typed graph planner, after which the plan is grounded against allowed schema fields, executed exhaustively, and put through preflight, postflight, answer-sanity, and evidence checks. Structural defects can trigger bounded replanning, but a clean empty result ends the process, so the system cannot keep searching until it finds an answer it prefers. The first protocol never invokes the understander, planner, evidence bundle, or repair loop, which stops an offline oracle check from being reported as an inference-time verifier and stops a component that was never called from taking credit for an item.

The experiments use a provenance-preserving graph built from UK procurement releases for 2022–2026 and a frozen 922-question compositional test. The typed-program model reaches 74.51% strict accuracy on the canonical surface, 38.50 points above the same untuned 8B base model and 25.70 points above a cloud planner, and loses only 1.30 points under independent surface naturalisation. Gains concentrate on overlap/exclusion and relational composition, while supplier concentration and disclosure/compliance stay much harder. A

separate balanced 260-item artifact puts the complete runtime at 86.15% against 70.38% untuned; we treat it as secondary evidence, because its stored outputs travel less well than the PACS artifacts.

Four things follow from this design. The first is the procurement graph itself, with a deterministic query-record projection and explicit entity-resolution, additivity, and coverage checks, which is what makes any denotation over it well defined. The second is the closed typed algebra together with the program-first supervision pipeline that teaches a compact model to write in it. The third is a runtime that keeps learned understanding and planning apart from grounding, exhaustive execution, evidence checks, bounded repair, and controlled non-answering. The fourth is PACS, a frozen compositional test with two surface channels, operator-family and depth diagnostics, answerability controls, and a published account of where its isolation and scoring procedures fall short. None of this guarantees that the source data are complete in the world, that stored organisation names are unique legal entities, or that passing an execution check proves semantic correctness. The claim we do make is testable: inside a declared frozen data universe, the computation a model proposes can be type checked, executed, traced, and rejected when its implemented preconditions fail.

2 Related Work

Neural-symbolic question answering makes an answer the denotation of an explicit program rather than free text (Liang et al., 2017), an idea KoPL develops into multi-hop, comparison, and counting operations (Cao et al., 2022) and question decomposition into smaller answerable units (Wolfson et al., 2020). Our algebra specialises it to contract records, separating record sets from sets of scalar values and adding semi-joins, anti-joins, set difference, grouped aggregation, and ranking. Reliability is then pursued at different points of the pipeline: token-level constrained decoding (Scholak et al., 2021), retrieval paired with constrained logical-form generation (Shu et al., 2022), and layered syntactic, ontology, and execution checks (Chen et al., 2021). We instead validate the completed typed object, then compile, ground, and re-check it at execution time. Execution by itself remains weak, since wrong programs often run cleanly (Wang et al., 2021); execution-guided refinement (Guo et al., 2023) and environment feedback (Huang et al., 2024) use that signal without trusting it, and so does our bounded repair.

Graph-based QA approaches the problem from the other side, retrieving or building a question-specific subgraph (Sun et al., 2018; 2019; Jiang et al., 2023b; Luo et al., 2024) or reading structured data through fixed interfaces (Jiang et al., 2023a), with an answer entity or path as the target. Exact counts and set differences instead need the complete denotation of an explicitly scoped program over a frozen backend, which is what turns ranking evidence into executing a query. Chain-of-Table is nearer to us, making intermediate tables part of the reasoning state (Wang et al., 2024); our traces do the same with typed sets, computed by a fixed evaluator rather than by the model.

How far such a parser generalises is what CFQ (Keysers et al., 2020) and COGS (Kim & Linzen, 2020) were built to measure, and reported gains on unseen compositions still leave a wide gap (Oren et al., 2020). PACS controls exact questions, tree hashes, and declared unseen shapes but has no divergence construction, so we make no CFQ-style claim for it. Our routing from a deterministic compiler to a learned planner follows grammar-plus-neural-fallback designs (Shaw et al., 2021), and our unsupported and empty-result controls follow work that separates unanswerable conditions from answerable ones (Patidar et al., 2023). On the data side, TheyBuyForYou integrates procurement releases, company data, and reconciliation into a public-procurement knowledge graph (Soylu et al., 2022); we share that motivation but ask how a model can propose a typed, auditable computation over such data, which leaves our graph as infrastructure rather than as evidence that the QA method is correct.

3 Background and Preliminaries

The input is a finite snapshot of OCDS releases D . A deterministic pipeline, described in Section 4.2, turns D into a knowledge graph $G = (V, E)$ and a query-record projection U_G . Each row $r \in U_G$ is keyed by a contract-node identifier and carries a controlled set of fields: stored buyer and supplier names, release year, procurement category and CPV code, selected dates, a monetary value and currency where available, an additivity flag, and provenance links. Throughout this paper, *record* means one row of U_G . The projection

Table 1: Guarantee boundary. Only oracle match is an offline answer-correctness measurement; none of the first four rows alone establishes real-world truth.

Term	Meaning in this paper
Static validity	The object belongs to the closed grammar and all operator input/output types compose; depth and node limits are respected.
Executability	The grounded object can be evaluated by the implemented backend without a structural or runtime failure.
Runtime verified	The implemented compile, preflight, execution, postflight, sanity, and evidence conditions required for release have passed.
Evidence supported	The answer is associated with the record/evidence identifiers emitted by execution under the implemented provenance policy.
Oracle match	The typed denotation equals the frozen benchmark oracle under the stated exact, set, Boolean, or numeric comparison rule.
Real-world truth	A claim about procurement activity outside the finite snapshot; this is not guaranteed by the system or benchmark.

keeps two notions of identity apart. Graph organisation nodes carry canonical identifiers produced by entity resolution, while query fields such as `buyer_name` hold stored strings taken from the first-party data, so a set of distinct stored names is a set of strings and we read it that way in every result below.

Let q be a question, $\mathcal{L}$ the closed typed algebra of Section 4.3, and $p \in \mathcal{L}$ a program. Static validation either assigns a root type,

$$\Gamma \vdash p : \tau, \quad \tau \in \{\text{Records, Values, Groups, Number, Value, Bool, Ranking}\}, \quad (2)$$

or returns a structured type error. Deterministic evaluation then yields an answer envelope $E_G(p) = (s, \tau, y)$, with execution status s and denotation y when $s = \text{OK}$. A surface renderer may verbalise y ; it may not add a factual value that y does not contain. Evaluation items divide into three operational statuses. An item is *answerable* when a program in $\mathcal{L}$ expresses the computation and its data preconditions hold in the snapshot; *unsupported* when the question needs an operation outside $\mathcal{L}$, such as a median, where an explicit abstention is the correct behaviour; and a *no-result* item when a well-formed scope has an empty denotation, where a faithful empty or zero answer is correct. The full runtime adds ambiguity and insufficient evidence as further states, and keeps all of them distinct, because each calls for a different explanation to the user.

Five notions of success are easy to conflate, and Table 1 keeps them apart so that one inference stays blocked: a program can be type valid and execute cleanly while expressing the wrong question. For an answerable item, strict correctness is

$$\mathcal{K}_{\text{strict}} = \mathcal{K}[\Gamma \vdash p : \tau] \mathcal{K}[E_G(p).s = \text{OK}] \mathcal{K}[\text{match}_{\tau}(E_G(p).y, y^*)], \quad (3)$$

where y^* is the frozen oracle and match_{τ} is type strict: sets compare as sets of strings, rankings preserve order and tie semantics, Booleans require Booleans on both sides, and numbers use the evaluator’s documented tolerance (Appendix B). Unsupported items score only on an explicit refusal of the operation, and no-result items follow their declared empty-result convention. Runtime verification is measured separately from Equation 3. Where both labels exist for an item, the informative diagnostic is their 2×2 table, which we report only for runs whose trace provenance is complete (Appendix D).

4 Method

4.1 Overview: two connected execution paths

The system contains a shared deterministic data substrate and two deliberately distinct language-model interfaces.

The **typed-program path** maps a question directly to either an algebra tree or an abstention object. It is used to generate program supervision and to measure compositional semantic-parsing capability. A tree is

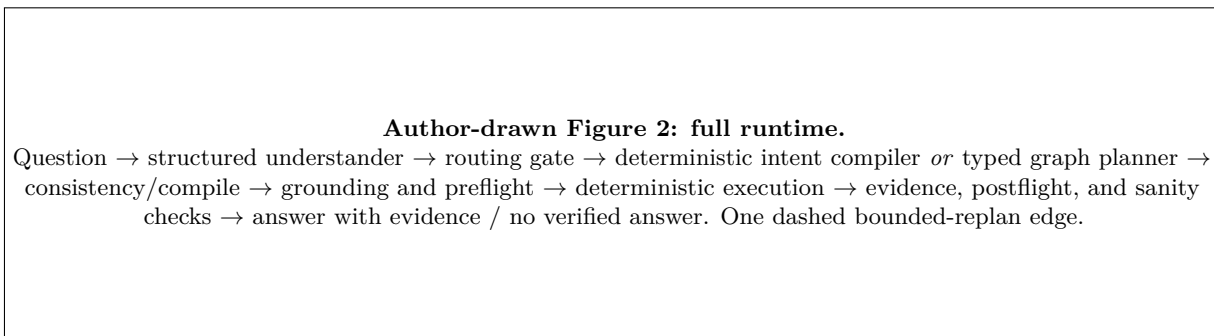


Figure 2: General full-runtime control flow. A directly compilable intent bypasses the graph model; a structured failure may trigger bounded replanning, after which the complete ground–execute–check sequence is repeated. The separate typed-program evaluation omits these deployment stages.

validated, executed over the query-record universe, and scored against an offline oracle. This path does not create an evidence bundle or call the deployment repair loop.

The **full runtime** first elicits a structured intent. A router then chooses a deterministic compiler whenever the intent can be compiled without either of two implemented fast-path vetoes; remaining questions are sent to a typed graph planner. Both branches converge on a graph/low-level specification that is grounded, preflighted, executed, and checked before an answer is released. A bounded feedback path is available only for eligible structured defects. Figure 2 is reserved for this general control flow, whereas Figure 1 must contain one concrete trace.

4.2 From OCDS releases to a queryable graph

The ingestion stage scans yearly gzip-compressed JSON Lines files. It skips empty and malformed lines rather than terminating a multi-year build, unwraps release packages, and flattens the fields needed by downstream extraction. OCDS publishes a new release when information about a contracting process changes ([Open Contracting Partnership, 2026](#)); consequently, multiple rows can share an Open Contracting ID (OCID). For each OCID, the implementation retains the row with the latest parseable release date and records all observed source years. This policy creates a current snapshot rather than a complete event history. It is deterministic for a fixed input ordering and explicitly differs from merging all release versions field by field.

The full pipeline entry point accepts an optional year subset. In the current implementation, partial runs must be directed to an explicit non-canonical output path; this prevents a small diagnostic run from silently replacing a complete artifact. The canonical full build preserves its original default path.

Entity resolution is conservative because a false merge affects every downstream join. Phase 1 uses official organisation identifiers when an observed identifier belongs to an accepted official scheme. A publication-system identifier (such as a GB-FTS identifier) can become an alias only when observations with the same OCID and normalised name share a party role and the complete candidate set points to exactly one official canonical identifier. Candidate identifiers are accumulated across observations before the uniqueness decision; a first-row match is not sufficient. This rule avoids merging two same-named parties that occur as buyer and supplier or that map to different official organisations.

Phase 2 resolves remaining parties in three deterministic passes: a government lookup table; exact normalised name plus region; and, when region is absent, exact normalised name. A resolved cluster receives a stable identifier derived from SHA-256. An observation whose name normalises to an empty string is retained as its own singleton instead of disappearing from the output. Jaro–Winkler similarity at a threshold of 0.92 is used only to emit review candidates; it never performs an automatic fuzzy merge. Thus the resolver favours false negatives over irreversible false positives.

The extraction stage produces normalised tables for tenders, lots, award criteria, awards, bid statistics, text evidence, and documents. For an award-level monetary value, the implementation uses the award value

Table 2: Core algebra operators. P is a predicate, f a field, m an aggregate metric, and $\bowtie_s$ an expression-valued membership predicate.

Family	Signature	Semantics
Filter	$\text{filter}_P : \text{Records} \rightarrow \text{Records}$	Select records
Project	$\text{values}_f : \text{Records} \rightarrow \text{Values}$	Distinct non-empty values
Cardinality	$\text{count} : \text{Records} \rightarrow \text{Number}$	Distinct contract count
	$\text{size} : \text{Values} \rightarrow \text{Number}$	Set cardinality
Aggregate	$\text{sum}_f : \text{Records} \rightarrow \text{Number}$	Decimal accumulation
Group	$\text{groupby}_{f,m} : \text{Records} \rightarrow \text{Groups}$	Keyed count or sum
Set	$\text{union/intersect/diff} : \text{Values}^2 \rightarrow \text{Values}$	Set algebra
Bridge	$\text{filter}_{f \bowtie_s e}$	Semi-/anti-join with $e : \text{Values}$
Rank	$\text{argext} : \text{Groups} \rightarrow \text{Value}$	Extreme group key
	$\text{top}_k : \text{Groups} \rightarrow \text{Ranking}$	Ordered key-value pairs
Scalar	$\text{combine} : \text{Number}^2 \rightarrow \text{Number/Bool}$	Arithmetic/comparison
Boolean	$\text{exists} : \text{Records} \rightarrow \text{Bool}$	Non-empty test

when present, otherwise a linked contract value, and finally a tender value as a fallback. Only award- and contract-sourced values are labelled `value_is_additive`; tender fallbacks are not. A numeric zero is a valid value and is not replaced by a later fallback. Currency is stored but not normalised.

Every extracted fact retains identifiers that link it to the OCID, release, award or contract, and source evidence where available. These links permit the runtime to return record-level provenance, but they do not prove that a publisher supplied every real-world fact.

The graph contains organisation, contract/notice, CPV, and evidence nodes. Buyer and supplier edges connect canonical organisations to contract nodes; category edges connect contracts to CPV nodes; and evidence edges connect source artifacts to contracts. Contract-node identifiers are deterministically formed from the OCID and award identifier. Before publication, the build checks primary-key uniqueness, referential integrity, JSON serialisability, additivity labels, and coverage thresholds. The frozen experimental graph contains 215,221contract nodes, 131,502organisation nodes, 3,870CPV nodes, and 535,731evidence nodes. Exact edge and coverage counts appear in Table 5.

The query backend materialises a flat record universe U_G from these Parquet nodes and edges. This representation intentionally makes the unit of `count` explicit: duplicate rows are deduplicated by `contract_node_id`. Buyer/supplier fields use the corresponding stored name projection. Grouping drops null and empty keys. This record view is the contract between the algebra and the graph; changing it changes benchmark semantics and therefore requires a new snapshot hash.

4.3 Closed typed algebra and deterministic execution

The algebra is *closed* in its operator inventory but permits open composition of those operators. Static validation limits a tree to depth 16 and 64 nodes and returns one of seven root types. Table 2 summarises the principal signatures; Appendix B provides the complete inventory and failure semantics.

Predicates support equality, membership, containment, existence, bounded numeric comparison, negation, disjunction, and expression-valued membership. The last form is the key to a bridge query. For example, if k is an anchor buyer name,

$$S = \text{values}_{\text{supplier_name}}(\text{filter}_{\text{buyer_name}=k}(U_G)), \quad (4)$$

$$B = \text{values}_{\text{buyer_name}}(\text{filter}_{\text{supplier_name} \in S}(U_G)), \quad (5)$$

$$A = B \setminus \text{values}_{\text{buyer_name}}(\text{filter}_{\text{buyer_name}=k}(U_G)). \quad (6)$$

The types make two common errors visible before execution: filtering `supplier_name` by a buyer anchor is a role error detectable only from semantics, but returning contract identifiers where a set of buyer values is expected also violates the declared answer signature.

Static validation checks tree shape and type composition, not the truth of the program or every domain guard. Field existence is checked during evaluation. Monetary training templates add an explicit `value_is_additive=true` predicate, but the generic `sum` operator does not itself prove that this predicate is present, and neither evaluator currently enforces a single currency. We therefore treat monetary results as subject to the limitation in Section 6.

The runtime algebra evaluator first calls the static validator, then evaluates the tree recursively over U_G . A record filter constructs a Boolean mask. An `in_expr` predicate first evaluates its child to a distinct value set and then applies membership to the parent field. Counts deduplicate contract identifiers. Distinct projection removes null and empty strings. Numeric sums deduplicate records and accumulate with `Decimal`; non-finite or non-numeric values fail the execution. Grouped counts deduplicate (group key, contract id) pairs. Rankings use a deterministic value-and-key ordering, and select operations fail rather than guess when more than one distinct answer remains.

A second evaluator is implemented separately for dataset generation and audit. A candidate training instance is retained only when the system evaluator and this independent implementation both return successful envelopes and their answers agree under the generator comparator. This reduces circularity between template generation and execution, although it is not formal verification: the two implementations can share assumptions, and the historical comparator had an overly permissive Boolean coercion rule (Section 6).

4.4 Program-first supervision

Training examples are sampled from the actual record distribution rather than generated as unanchored text. A sampler selects concrete buyers, suppliers, years, CPV codes, categories, and other field values; instantiates a typed program; renders a natural-language instruction; and executes the program. We reject a sample if (i) the tree fails static validation, (ii) either evaluator fails, (iii) the evaluators disagree, (iv) the answer is degenerate for its declared type, or (v) the question string has already been accepted. Degeneracy rules exclude zero-valued numeric examples, empty value sets, singleton rankings, and missing scalar answers. These rules are a data-construction heuristic, not a statement that zero is intrinsically invalid at inference time.

The resulting LLaMA-Factory export contains 12,414 training and 253 validation examples. Each target is JSON containing either a typed tree or, for unsupported evaluation controls, an abstention object. The specialised parser adapts Qwen3-8B (Yang et al., 2025) with 4-bit QLoRA (Detrmers et al., 2023); the optimiser settings are given in Section 5.2. The held-out synthetic loss is reported only as an optimisation diagnostic because the export contains duplicates and two exact train/validation overlaps.

4.5 Understanding, conditional planning, and grounding

The full runtime begins by retrieving a compact schema context and requesting a structured intent program. The intent records the question shape, answer signature, source record constraints, intermediate entity-set roles, target constraints, and return operation. Provider-enforced JSON schema output is used when available, with a text/JSON fallback for older clients. A malformed briefing receives one cheap structural retry; an unparseable result becomes a typed planning failure rather than a factual answer.

If the object is a recognised intent program, a deterministic compiler maps it to a graph plan. The compiled fast path is accepted unless compilation fails or one of two narrow semantic vetoes fires: an unsupported cue that could be converted into an invented answer, or a mismatch between an entity-set count and a record-count question. The implementation intentionally does not run the raw-planner consistency checker over every compiled graph, because checks calibrated for raw model payloads produced false positives on deterministic compiler output.

Questions that do not pass this gate invoke the Step-2 graph planner. The planner emits typed variables, dependencies, operation units, and a return specification. A consistency checker tests the payload against the question, and the compiler normalises and attaches low-level query specifications. Structural com-

pile/consistency failures may trigger a small number of fresh plan samples. An executable but semantically wrong plan is not detectable from structure alone and is not resampled using an oracle at inference time.

This conditional design is related to high-precision grammar plus neural fallback systems (Shaw et al., 2021). A complete routing ablation would compare compiler-only, always-planner, and conditional modes under the same model and budget. The present artifact set does not contain that complete three-way experiment, so the paper describes the implemented mechanism but does not attribute the entire end-to-end gain to routing.

Grounding maps aliases to canonical backend fields, normalises values, resolves confident organisation mentions, and rejects fields outside the allowed backend profile. It also attaches operation-specific guards such as the additivity predicate used for monetary operations. A preflight phase checks schema support and query preconditions before any answer is produced.

For a graph plan, variables are executed in dependency order. A source record-set variable applies its grounded constraints. An entity-set variable emits distinct values from its source. The emitted set is then bound to the appropriate field of a dependent target variable—for example, supplier values are bound to `supplier_name`, whereas a record-set intersection binds contract identifiers. Variables at the same dependency level may execute in parallel; a terminal operation computes the requested count, set, comparison, or other supported answer. Each variable trace stores its status, grounded constraints, emitted field, output size, and answer summary. The executor is the sole factual authority: the planner is never asked to invent the terminal value.

4.6 Release checks and bounded repair

Execution returns record and evidence identifiers in an `EvidenceBundle`. Postflight checks inspect operation-specific properties of the returned rows, while answer-sanity checks detect incompatible shapes, non-finite numbers, and other release defects. An evidence verdict summarises whether the implemented support requirement is satisfied. A released `AnswerCard` contains the denotation or its verbalisation, confidence label, evidence identifiers, and limitations. If a required check fails, the runtime produces no verified answer.

These checks provide operational guarantees, not an oracle. They can establish that a program uses allowed fields, executes over the declared scope, and returns linked records. They cannot establish that the plan captures the user’s intended semantics or that the publisher’s data are complete. We therefore reserve *correct* for the offline oracle metric and use *runtime verified* only in the sense defined by Table 1.

Algorithm 3 summarises the implemented control logic. Repair is eligible when a structured defect supplies actionable feedback: compilation or consistency rejection, failed preflight/postflight checks, an execution/verifier failure, or a no-result plan containing a literal not traceable to the question. The feedback object records the failure stage, failed checks, grounding information, and invented literals. A new plan is then grounded, executed, and checked from the beginning. No repaired answer is accepted merely because its surface form looks better.

Conversely, if a fully grounded query executes to no results and every literal is traceable to the question, the empty result is terminal. Replanning in this case would amount to answer-shopping. Unsupported operations, unresolved ambiguity, exhausted repair budgets, and insufficient evidence all terminate with a controlled non-answer.

5 Experiments

Four questions drive the evaluation. Does typed-program supervision improve strict compositional execution accuracy over the same untuned base model and a strong cloud planner (RQ1)? Does the specialised parser hold that accuracy under an independently generated surface form (RQ2)? Which operator families and composition depths stay hard (RQ3)? And does the complete understand–plan–ground–execute–check runtime improve on its untuned counterpart (RQ4)?

Algorithm 1: Full question-answering runtime

1. Obtain structured intent $i \leftarrow \text{UNDERSTAND}(q, \text{schema})$.
2. If i compiles and no fast-path veto fires, set $H \leftarrow \text{COMPILE}(i)$; otherwise obtain and consistency-check $H \leftarrow \text{PLAN}(q, i)$.
3. For each attempt up to the fixed budget: ground all variables and literals; run preflight; execute variables in dependency order; construct evidence; run postflight, sanity, and evidence checks.
4. If all release conditions pass, return an answer card whose factual value is the executor denotation.
5. If the defect is repair-eligible, send structured feedback to the planner and return to Step 3. Otherwise return the corresponding unsupported, ambiguous, no-result, or insufficient-evidence non-answer.

Figure 3: Runtime control procedure. Repair changes the plan, never the factual answer directly, and repeats the complete deterministic path.

5.1 Datasets and metrics

Every execution below runs over one Parquet knowledge-graph snapshot and query-record projection, whose node counts and edge coverage are in Table 5. Supervision comes from the program-first pipeline of Section 4.4, sampled from the same frozen record universe: 12,414 training and 253 validation instances, each a typed tree or an abstention object that has survived static validation, agreement between the two independent evaluators, the non-degeneracy rules, and question-level deduplication. The export also contains exact duplicates and two exact train/validation overlaps, so its held-out loss is an optimisation diagnostic and not a generalisation estimate.

PACS tests combinations of closed-algebra primitives rather than broad world knowledge. Its seven capability families are the rows of Table 4; its levels run from an atomic computation (L1) through two or three combined operations (L2) to nested set membership, filtered comparisons, set algebra, or group composition (L3). Gold trees are instantiated from family templates over sampled anchors and kept only when they validate, execute identically in both implementations, and pass a family-specific non-degeneracy rule, after which whole intent clusters are assigned to development or test together (Appendix C). The frozen test holds 922 items per surface: 694 answerable, 138 unsupported, and 90 no-result; 279 at L1, 339 at L2, and 304 at L3; 431 seen and 491 declared unseen shapes. Family sizes range from 67 to 149, so overall accuracy is a micro average with macro-family accuracy beside it.

Each intent is evaluated on two surfaces sharing one gold program and oracle. Channel A renders the semantics deterministically and independently; Channel B is a separately naturalised question that must keep the required literals and pass a logic gate, so A–B is a paired robustness measurement over a fixed computation. Sealing checks exact normalised questions, gold tree hashes, and declared unseen shapes against the training export, but its Channel-B near-duplicate screen is subsampled and one of the five documented gates has no separate implementation. PACS is therefore a *mechanically screened compositional test*, not a proven zero-leakage split.

The primary metric is strict accuracy from Equation 3; the supplementary *safe accuracy*, which also credits literal-faithful empty-result executions, is defined in Appendix D. Tables give exact numerators and denominators, and parenthesised intervals are 95% Wilson score intervals over the fixed test items.

5.2 Implementation and baselines

The specialised parser adapts Qwen3-8B with 4-bit QLoRA at rank 64, scaling $\alpha = 128$, and dropout 0.05, with all target modules adapted. Training runs for two epochs at per-device batch size 2 with gradient accumulation 8, an effective batch of 16, learning rate 10^{-4} under cosine decay with 3% warm-up, a 4096-token cutoff, and bfloat16 computation. Every headline evaluation uses one decode per item at temperature zero, with no guided JSON grammar and no reflection or repair, which keeps the local and cloud arms comparable; because these are single deterministic decodes, no interval reported here estimates sensitivity to checkpoint,

Table 3: PACS strict execution accuracy. All Channel-A rows use the same 922 cases and a single temperature-zero free decode. “Valid tree” is reported for diagnosis only: explicit abstentions do not emit a tree, and validity does not imply correctness.

System	Surface	Correct	n	Strict acc.	95% Wilson CI	Valid tree
Untuned Qwen3-8B	A	332	922	36.01	[32.97, 39.16]	77.77
Cloud planner	A	450	922	48.81	[45.59, 52.03]	52.28
Typed-program Qwen3-8B	A	687	922	74.51	[71.60, 77.22]	71.58
Typed-program Qwen3-8B	B	675	922	73.21	[70.26, 75.97]	70.50

prompt, provider, or hardware. The full runtime is evaluated with a repair budget of one replan. Endpoints, model revisions, prompt hashes, and the commands that regenerate every artifact are in Appendix A.

All PACS systems receive the same algebra description and one-question prompt. **Untuned 8B** is Qwen3-8B with no task-specific fine-tuning; the **cloud planner** is **grok-4-1-fast-non-reasoning** under the same interface and algebra contract; **typed-program 8B** is the QLoRA adapter above. Only the specialised parser was run on Channel B, so Channel A carries the three-system comparison. Two baselines sit outside the PACS tables. A **retrieval** baseline indexes the record projection with TF-IDF and passes the top- k records to an untuned reader, plain at $k=10$ and field-boosted at $k=40$, standing for the evidence mechanism our method replaces (Section 5.5). The end-to-end comparison uses a balanced 260-item artifact of 13 categories with 20 cases each, comparing the full local runtime, the same stack without specialised adaptation, and a stored cloud-teacher run. No component ablation of routing, verification, and repair was run; Section 6 states what that costs the attribution.

5.3 Main results

Table 3 answers RQ1. The typed-program model solves 687/922 cases, 38.50 points above the same untuned base model and 25.70 points above the cloud planner, and its Wilson interval overlaps neither baseline.

The valid-tree column moves against accuracy. Ranked by static validity alone the untuned base model comes first, 6.19 points above the specialised parser, while trailing it by 38.50 points on strict accuracy. A well-typed program can still reverse two roles, project the wrong field, or express the wrong scope, and an explicit abstention is a scoreable final action that emits no tree at all: the guarantee boundary of Table 1, visible as a measurement.

For RQ2 the specialised model falls 1.30 points from Channel A to Channel B, with macro-family accuracy down from 72.07% to 70.82%. The parser is not reproducing one fixed wording, then, though this is also not unrestricted linguistic robustness, since both channels preserve the same literals under explicit logic gates. The largest per-family decline is 3.38 points on relational composition (F6), while supplier concentration (F3) improves by 2.96 points. A paired significance test is left out of this draft: the strict Boolean-matching fix has to reach every scorer first, and the paired decisions must then be regenerated from the frozen outputs.

5.4 Results by family and depth

Table 4 answers RQ3. Overlap and exclusion (F5) is the strongest family at 94.49%, more than 60 points above the base model, with large gains also on cross-buyer comparison (F4) and relational composition (F6). Supplier concentration (F3) and disclosure/compliance (F7) stay near 50% and 42%, and on both the cloud model matches or slightly beats the specialised parser. The micro average hides this, since F7 contributes only 67 of 922 cases.

Depth behaves less regularly than family: 92.11% at L1, 61.06% at L2, and 73.36% at L3, where the L2 dip follows from how families and status types distribute across levels rather than from two-step programs being intrinsically harder than three-step ones. Declared seen and unseen shapes score 74.01% and 74.95%. That near equality is encouraging within its construction: with no atom/compound-divergence design and a

Table 4: Strict accuracy by PACS family. Exact correct counts are given in parentheses. Surface B is available only for the specialised parser.

	Family (n)	Untuned A	Cloud A	Typed A	Typed B
F1	Spending/volume (148)	41.89 (62)	62.84 (93)	84.46 (125)	81.08 (120)
F2	Temporal change (149)	57.72 (86)	72.48 (108)	83.22 (124)	81.88 (122)
F3	Supplier concentration (135)	19.26 (26)	51.85 (70)	51.11 (69)	54.07 (73)
F4	Cross-buyer comparison (147)	39.46 (58)	44.22 (65)	81.63 (120)	79.59 (117)
F5	Overlap/exclusion (127)	32.28 (41)	33.07 (42)	94.49 (120)	93.70 (119)
F6	Relational composition (149)	28.19 (42)	28.86 (43)	67.79 (101)	65.10 (97)
F7	Disclosure/compliance (67)	25.37 (17)	43.28 (29)	41.79 (28)	40.30 (27)
	Macro family	34.88	48.09	72.07	70.82

subsampled near-duplicate screen, it evidences generalisation to a declared structural holdout, not CFQ-style compositional generalisation. Appendix D holds the full depth and exposure tables.

For RQ4, the full local runtime solves 224/260 cases (86.15%, 95% Wilson CI [81.43, 89.83]), against 183/260 (70.38%) untuned and 192/260 (73.85%) for the stored cloud-teacher run. Category gains concentrate on bridge joins (14/20 versus 2/20), comparisons (17/20 versus 7/20), and categorical questions (13/20 versus 8/20), the same profile as the PACS families. All three systems score 20/20 on the sum category, but the underlying data contain unnormalised non-GBP additive rows, so that column is not evidence of currency-correct aggregation. Its run manifest is less complete than the PACS one, which is why we treat it as secondary evidence; the per-category table is in Appendix D.

5.5 Analysis

Figure 1 follows one bridge question through all three mechanisms: *how many contract notices were published by buyers who have awarded a contract to Edmundson Electrical Limited?* Over U_G it denotes 23 records naming that supplier, 16 buyer values projected from them, and 4,257 notices published by those buyers. The two failures differ in kind. Retrieval at $k=40$ returns all 23 first-hop records, so its answer of 14 is not a recall failure: the notices being counted were never retrieved and could only be counted by retrieving all 4,257. The untuned model instead emits `count(filter(supplier_name = anchor))`, which passes static validation with root type `Number` and executes without error to 23, the size of the first hop. Both return a plausible number; neither expresses the question. The 16 buyer values also carry the entity caveat in miniature, since they include both *LEEDS BECKETT UNIVERSITY* and *the Education Authority*: stored strings, not resolved legal entities.

The aggregate gain decomposes the same way. The specialised parser’s 687 strict successes are 549 correct executed answers and 138 correct explicit abstentions, against 291 and 41 for the untuned model, which also answers 114 out-of-grammar cases and emits 104 invalid trees. Typed-program supervision therefore changes both semantic coverage and the willingness to use the unsupported action; syntax repair alone would not produce this pattern, since the base model already emits type-valid trees at a high rate while encoding the wrong role, projection, or composition in many of them. The cloud planner sits between the two, with 324 correct answers and 126 correct abstentions against 161 invalid trees, 57 unparseable outputs, and 25 API errors. Counting service errors as failures is the right convention for a single-run comparison, but it blends model capability with provider availability.

Strict scoring also leaves something on the table: the specialised Channel-A run contains 73 literal-faithful empty-result executions that receive no credit, because those non-answerable controls require an explicit abstention, and crediting them would raise the score to 82.43%. That gap measures whether the model avoided an unsupported factual answer under a broader operational policy, which is a different question from exact-answer accuracy, and a production interface has to choose between them: a user asking for a count over an empty scope may well prefer “0”, whereas a missing-operator question needs to be told that the operation is unsupported.

6 Discussion and Limitations

6.1 Discussion

The 86.15% system result is consistent with the intended benefit of specialised planning inside an execute-and-check runtime, and its category gains on bridge, comparison, and set questions line up with the PACS families. The comparison still cannot separate the understander, router, deterministic compiler, graph planner, verifier, and repair loop, so we report the full-stack difference without assigning it to any component. The runtime’s more durable property is inspectability: a failure localises to an unparseable intent, a routing veto, graph inconsistency, a grounding error, a preflight failure, an empty intermediate set, a postflight defect, or an evidence decision. That is worth having even where final accuracy is unchanged, and it is best called *auditable* rather than interpretable, since the learned understander and planner still do not explain their own decisions.

One pattern cuts across every result above. The bridge case study projects 16 stored buyer strings rather than 16 resolved organisations, and a correct program over the unnormalised currency column can return a reproducible total whose label is wrong. Both pass the benchmark, because the benchmark inherits its denotation from the same projection the model is trained on. Entity and currency semantics are not downstream data-engineering chores; they set the target the model is scored against, which is where the limitations begin.

6.2 Limitations

The data are a 2022–2026 snapshot of published OCDS releases, so exhaustive execution is exhaustive over the declared record universe and not over all procurement activity: late, incorrect, or absent disclosures stay absent, and keeping one release per OCID discards event history. Conservative entity resolution leaves unresolved aliases in the stored query fields, so a value-set answer is a set of strings. The source code now handles role-safe publication-system aliases, blank-name singletons, and globally ambiguous candidates, but the frozen graph predates those patches, and results stay bound to its hash until the pipeline is rebuilt and the benchmark reverted.

Monetary semantics are weaker still. Of 176,002 rows marked additive, 1,059 carry a non-GBP currency, and neither the algebra nor the runtime converts currency or hard-enforces a currency filter while some prompts describe totals as GBP. Monetary results can therefore mix currencies, and no sum-related number in this paper is currency-correct financial analysis.

On the supervision and evaluation side, the training export contains exact duplicates and two exact train/validation overlaps, which is why its validation loss is only an optimisation diagnostic. PACS removes exact question and tree-hash collisions and controls declared shape exposure, but a subsampled near-duplicate screen and one unimplemented gate make it a mechanically screened holdout rather than proven zero leakage. Several generation and scoring helpers also used Boolean coercion that can equate a Boolean with a truthy non-Boolean; a strict audit cleared the stored PACS typed-program A run and we exclude the affected legacy result sets, but one shared type-strict comparator should reach every scorer before any new headline number.

Every headline evaluation is a single temperature-zero decode, and Wilson intervals cover test-item uncertainty alone, not training seeds, checkpoints, prompts, providers, or serving stacks. There is also no same-budget ablation over compiler-only, always-planner, conditional routing, verification, and repair. This is the sharpest gap in the present evidence, since the result supports the full configuration and can attribute nothing to a single module, and closing it needs no new data: the same frozen snapshot, run once under each variant, would settle it.

Finally, the checks reject invalid objects, unsupported fields, malformed executions, and insufficient evidence, and that is all they do. They cannot show that a valid program represents the question asked, or that an accepted answer is complete in the world, so runtime-verified rate is never hallucination-free correctness. Several end-to-end adapters and summaries also resolve through a machine-specific absolute link (Appendix A), which is why PACS carries the primary claims. The study is English-language and UK-specific, and exact

program accuracy says nothing about whether practitioners trust the evidence returned, which would take a user study of task completion, error detection, and abstention explanations.

6.3 Broader impact

The source records describe public contracting activity, but even simple system outputs can affect the reputation of named organisations. The pipeline is built for data exploration and audit support, not for automated allegations, supplier scoring, or procurement decisions. An answer should travel with its data scope, provenance, and limitations, and missing records, unresolved aliases, and unnormalised currencies must never be converted silently into claims about misconduct. A record being public also does not remove the need to review licensing, personal-data fields, and downstream use before redistribution.

7 Conclusion

We treated public-procurement question answering as typed semantic parsing followed by deterministic execution rather than as direct factual generation. A provenance-preserving graph supplies the finite query universe, a closed algebra expresses set, bridge, aggregate, comparison, and ranking computations, and the deployment runtime adds structured understanding, conditional planning, grounding, evidence checks, bounded replanning, and controlled non-answering. Typed-program adaptation raises strict PACS accuracy from 36.01% to 74.51% and holds 73.21% under independent surface naturalisation; on a separate system snapshot the full local runtime reaches 86.15% against 70.38% untuned.

The audits matter as much as the gains. Type validity is not semantic correctness, runtime verification is not an oracle, exhaustive execution is exhaustive only inside a frozen snapshot, and unresolved aliases and currencies quietly change what a correct program denotes. The next defensible step is not a larger headline experiment but a versioned rebuild: enforce strict answer types and currency semantics, publish portable manifests, and measure routing, verification, and repair under controlled ablations. That is what would turn an executable prototype into a reproducible account of when typed, evidence-linked question answering succeeds and when it should decline to answer.

References

- Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyiu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. KQA pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6101–6119, 2022. doi: 10.18653/v1/2022.acl-long.422. URL <https://aclanthology.org/2022.acl-long.422/>.
- Shuang Chen, Qian Liu, Zhiwei Yu, Chin-Yew Lin, Jian-Guang Lou, and Feng Jiang. ReTraCk: A flexible and efficient framework for knowledge base question answering. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pp. 325–336, 2021. doi: 10.18653/v1/2021.acl-demo.39. URL <https://aclanthology.org/2021.acl-demo.39/>.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- Wangzhen Guo, Linyin Luo, Hanjiang Lai, and Jian Yin. From parse-execute to parse-execute-refine: Improving semantic parser for complex question answering over knowledge base. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 11771–11780, 2023. doi: 10.18653/v1/2023.emnlp-main.720. URL <https://aclanthology.org/2023.emnlp-main.720/>.
- Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction.

-
- In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5014–5035, 2024. doi: 10.18653/v1/2024.acl-long.274. URL <https://aclanthology.org/2024.acl-long.274/>.
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9237–9251, 2023a. doi: 10.18653/v1/2023.emnlp-main.574. URL <https://aclanthology.org/2023.emnlp-main.574/>.
- Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. UniKGQA: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. In *International Conference on Learning Representations*, 2023b. URL <https://openreview.net/forum?id=Z63RvyAZ2Vh>.
- Daniel Keysers, Nathanael Sch"arli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, Dmitry Tsarkov, Xiao Wang, Marc van Zee, and Olivier Bousquet. Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SygcCnNKwr>.
- Najoung Kim and Tal Linzen. COGS: A compositional generalization challenge based on semantic interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 9087–9105, 2020. doi: 10.18653/v1/2020.emnlp-main.731. URL <https://aclanthology.org/2020.emnlp-main.731/>.
- Chen Liang, Jonathan Berant, Quoc Le, Kenneth D. Forbus, and Ni Lao. Neural symbolic machines: Learning semantic parsers on Freebase with weak supervision. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23–33, 2017. doi: 10.18653/v1/P17-1003. URL <https://aclanthology.org/P17-1003/>.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/3e2aeb66481dd63a32421bf032b70384-Abstract-Conference.html.
- Open Contracting Partnership. Open contracting data standard 1.1.5: Reference. Online documentation, 2026. URL <https://standard.open-contracting.org/latest/en/schema/>. Accessed 12 August 2026.
- Inbar Oren, Jonathan Herzig, Nitish Gupta, Matt Gardner, and Jonathan Berant. Improving compositional generalization in semantic parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2482–2495, 2020. doi: 10.18653/v1/2020.findings-emnlp.225. URL <https://aclanthology.org/2020.findings-emnlp.225/>.
- Mayur Patidar, Prayushi Faldu, Avinash Singh, Lovekesh Vig, Indrajit Bhattacharya, and Mausam. Do I have the knowledge to answer? investigating answerability of knowledge base questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10341–10357, 2023. doi: 10.18653/v1/2023.acl-long.576. URL <https://aclanthology.org/2023.acl-long.576/>.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779. URL <https://aclanthology.org/2021.emnlp-main.779/>.
- Peter Shaw, Ming-Wei Chang, Panupong Pasupat, and Kristina Toutanova. Compositional generalization and natural language variation: Can a semantic parsing approach handle both? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 922–938, 2021. doi: 10.18653/v1/2021.acl-long.75. URL <https://aclanthology.org/2021.acl-long.75/>.

-
- Yiheng Shu, Zhiwei Yu, Yuhang Li, Björn F. Karlsson, Tingting Ma, Yuzhong Qu, and Chin-Yew Lin. TIARA: Multi-grained retrieval for robust question answering over large knowledge base. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 8108–8121, 2022. doi: 10.18653/v1/2022.emnlp-main.555. URL <https://aclanthology.org/2022.emnlp-main.555/>.
- Ahmet Soylu, Oscar Corcho, Brian Elvesaeter, Carlos Badenes-Olmedo, Tom Blount, Fernando Yedro Martínez, Mateja Kovacic, Marco Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Tomasz C. Lech, Dumitru Roman, and Jens Lehmann. Theybuyforyou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022. doi: 10.3233/SW-210442. URL <https://doi.org/10.3233/SW-210442>.
- Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 4231–4242, 2018. doi: 10.18653/v1/D18-1455. URL <https://aclanthology.org/D18-1455/>.
- Haitian Sun, Tania Bedrax-Weiss, and William Cohen. PullNet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 2380–2390, 2019. doi: 10.18653/v1/D19-1242. URL <https://aclanthology.org/D19-1242/>.
- Bailin Wang, Mirella Lapata, and Ivan Titov. Learning from executions for semantic parsing. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2747–2759, 2021. doi: 10.18653/v1/2021.naacl-main.219. URL <https://aclanthology.org/2021.naacl-main.219/>.
- Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f53fd88a4340063ecd258c0ae9948b40-Abstract-Conference.html.
- Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark. *Transactions of the Association for Computational Linguistics*, 8:183–198, 2020. doi: 10.1162/tacl_a_00309. URL <https://aclanthology.org/2020.tacl-1.13/>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.

A Reproducibility Snapshot

A.1 Code and artifact state

The repository base commit inspected for this draft is `d15ec0a3bd5b5f295f9828e44067094b73ab2cd7`. The working tree also contains uncommitted correctness patches and staged data artifacts; the commit alone is therefore not an exact reproduction capsule. Before submission, the authors must create a clean tagged revision and record the patch, Python environment lock, CUDA/driver versions, model revisions, and command lines. No new model training or batch inference was run to prepare this draft.

Per-item PACS predictions and summaries are stored under `data/qa/compose_probe_v1/`. The primary summaries are `summary_pacstest_base_a.json`, `summary_pacstest_teacher_a.json`, `summary_pacstest_v3_a.json`, and `summary_pacstest_v3_b.json`. The corresponding JSON Lines files retain the raw model response, parsed tree, execution answer, oracle, outcome, and correctness flag.

Table 5: Frozen knowledge-graph snapshot and contract-level edge coverage. Coverage is structural and is not a correctness rate.

Block	Item	Count	Share
Graph nodes	Contract/notice	215,221	
	Organisation	131,502	
	CPV	3,870	
	Evidence	535,731	
Contract coverage	Buyer edge	215,218	99.999%
	Supplier edge	204,186	94.87%
	Category edge	164,691	76.52%
	At least one evidence edge	215,202	99.99%

Table 6: Content hashes for the primary frozen inputs.

Repository path	SHA-256
data/qa/pacs_v1/test_channel_a.jsonl	1869558ad2764b92888df47b093dfc2f f693d2dfdd6544f1fa3734d017cf2bb
data/qa/pacs_v1/test_channel_b.jsonl	36100c4bfb7b34ef6fe1d72c62d33a62 d069501aad97c0e28a172345a901339b
data/training/llamafactory_compose_v3/compose_sft_train.json	a80945184aeb9531321b26ef57286656 8707eeba10b7dc56ec34f8e35f00a20d
data/training/llamafactory_compose_v3/compose_sft_val.json	7cba3d43610a90216e5d7655319178df 3a66f1728d1133b704808b0a8ba2a899
data/kg/nodes/contract_nodes.parquet	246caad93e1c8b184d3714a280190efb 3c238f4cab73cd3bf1f36b38de307bf1
data/kg/nodes/org_nodes.parquet	42f9542833275ea7a57feb2cc4f4e64 a507513f35b2e420a449d938b68b867af

The 260-item runtime summaries are under `outputs/eval/matrix_v2/fully-local-qwen/`, `.../cicada-qwen3-zeroshot/`, and `.../teacher_r3/`. These paths contain an internal historical label and currently resolve through a machine-specific absolute symlink; that label is not used as the method’s paper name. A release must replace the absolute link with content-addressed manifests and publish model, prompt, environment, and evaluator hashes, which is the condition under which the 260-item runtime result could be promoted from a frozen artifact to primary evidence.

A.2 Pipeline entry points

A full deterministic data build follows this order:

```
.venv/bin/python -B pipelines/01_ingest.py
.venv/bin/python -B pipelines/02_er_phase1.py
.venv/bin/python -B pipelines/03_er_phase2.py
.venv/bin/python -B pipelines/04_extract.py
.venv/bin/python -B pipelines/40_build_kg.py
.venv/bin/python -B pipelines/41_validate_kg.py
```

Partial year/sample runs require explicit alternate outputs in the current code and must never be used to overwrite the canonical artifact. Exact raw-data acquisition commands and licences remain to be added to the release manifest.

Program supervision is built with `scripts/build_compose_train.py`, exported with the compose export script, and trained with `configs/training/qwen3_8b_compose_sft_v3_qlora.yaml`. The stored training summary reports train loss 0.0171394, evaluation loss 0.0096943, and runtime 8270.93 seconds. These losses are diagnostics only because the exported split is not fully deduplicated.

PACS generation uses, in order, `scripts/pacs/generate.py`, `render_channels.py`, `surface_independent.py`, `naturalize.py`, and `seal.py`. Evaluation uses `scripts/run_compose_probe_eval.py`. A release command must record the serving endpoint, ex-

act model revision, API date/version, prompt hash, maximum tokens, concurrency, temperature, guided-decoding flag, reflection budget, and scorer revision.

A.3 Checklist before final submission

The final artifact should contain: (i) a clean source tag and dirty-tree check; (ii) raw-data and KG manifests; (iii) model/adaptor hashes or durable release links; (iv) exact prompts and JSON schemas; (v) strict scorer unit tests covering Boolean versus numeric values; (vi) route, repair, and failure counts; (vii) random seeds and repeated-run policy; (viii) hardware and software versions; and (ix) a one-command script that regenerates every table from JSON/Parquet without copying numbers by hand.

B Complete Algebra Contract

Table 7: Complete expression-node inventory in the current validator.

Node	Input type(s)	Output
<code>filter</code>	predicates over U_G	Records
<code>values</code>	Records, field	Values
<code>count</code>	Records	Number
<code>size</code>	Values	Number
<code>sum</code>	Records, field	Number
<code>exists</code>	Records	Bool
<code>select</code>	Records, field	Value
<code>extreme_rows</code>	Records, field, argmin/argmax	Records
<code>extreme</code>	Records, field, argmin/argmax	Value
<code>groupby</code>	Records, key, count/sum metric	Groups
<code>argext</code>	Groups, argmin/argmax	Value
<code>top</code>	Groups, $1 \leq k \leq 50$	Ranking
<code>num</code>	numeric non-Boolean literal	Number
<code>combine</code>	Number, Number	Number or Bool
<code>vcompare</code>	Value, literal	Bool
<code>setop</code>	Values, Values	Values
<code>gcombine</code>	Groups, Groups	Groups
<code>keys_where</code>	Groups, comparator, threshold	Values

Base predicates are `eq`, `in`, `contains`, `exists`, `gte`, and `lte`. `not` negates one predicate, `any` disjoins two or more predicates, and `in_expr` evaluates a `Values` child and applies membership or negated membership to a field. Numeric `combine` operators are `diff`, `ratio`, and `add`; Boolean comparisons are `gt`, `lt`, `ge`, `le`, and `eq`. Set operators are union, intersection, and difference.

The validator enforces compositional types, required arguments, a maximum depth of 16, a maximum of 64 nodes, and $k \leq 50$. It does not restrict every field to the runtime schema and it does not make the additivity or currency guard intrinsic to `sum`. Those are separate domain contracts. Runtime failure modes include unknown fields, non-finite/non-numeric sums, no result, and multiple distinct values for a scalar selection. For `extreme_rows`, all tied rows at the extremum are retained; downstream scalar selection therefore fails on unresolved ties instead of choosing an arbitrary row.

The comparator used by Equation 3 is type strict. Set-valued answers compare as sets of strings; rankings preserve their defined order and tie semantics; Booleans require Boolean values on both sides, so Python’s subtyping of `bool` under `int` must never make `True` equivalent to numeric one; numeric answers use the evaluator’s documented tolerance. Section 6 records where legacy helpers violated the Boolean rule and which result sets are excluded as a consequence.

Counts and grouped counts deduplicate contract identifiers. Projections return sorted distinct non-empty strings. Grouped keys exclude null and empty strings. Numeric sums use decimal accumulation before conversion to the JSON answer representation. Ranking ties are broken deterministically by the key string after the numeric score.

Table 8: PACS capability families and intended composition at each level.

ID	Capability	Typical L1–L3 progression
F1	Spending/volume	Filtered count or additive sum; combined scopes; nested supplier-conditioned scope.
F2	Temporal change	Existence/count by year; comparison or difference; grouped temporal extreme.
F3	Supplier concentration	Supplier count; grouped top supplier; buyer-conditioned supplier grouping.
F4	Cross-buyer comparison	Buyer scope; same-category compare/difference; differently filtered two-sided comparison.
F5	Overlap/exclusion	Supplier set; union/intersection/difference; record count conditioned on a derived set.
F6	Relational composition	Buyers of supplier; year-filtered buyer set; buyer–supplier– buyer bridge with anchor exclusion.
F7	Disclosure/compliance	Missing-field count; buyers with at least one missing field; grouped threshold/perfect-coverage condition.

C PACS Construction Details

The 922-case test contains 148/149/135/147/127/149/67 examples from F1–F7. Depth counts are 279 L1, 339 L2, and 304 L3. Shape exposure is 431 seen and 491 declared unseen. Operational status counts are 694 answerable, 138 unsupported, and 90 no-result cases. Because status variants travel with their base intent cluster, they do not cross the development/test split.

The sealing implementation uses seed 20260719. It drops an entire cluster if any of its three surface candidates exactly collides with a normalised training instruction, relabels status variants from their actual tree shape, and checks the exact question, tree hash, and unseen-shape conditions. For the naturalised channel it computes character-trigram Jaccard similarity against `train_tris[:,7]`; this performance shortcut is disclosed because it weakens an otherwise all-pairs interpretation of the gate. The console message currently says that G3 passes even though no separate G3 counter is evaluated. The paper wording follows code, not that message.

The audit sample selects up to five naturalised instances per family–depth cell and additional status examples with the same seed. Before release, a human audit should record literal preservation, semantic equivalence, answerability, and whether any generated wording is unnatural or discloses a template artifact. The current repository contains the sample but not a complete signed human-audit report.

D Full Diagnostic Results

Table 9: PACS outcome summary. Safe accuracy adds faithful multi-answer/empty-result outcomes to strict correctness; it is not a second exact-answer metric.

System/surface	Strict	Safe	Abstain	Answered	Faithful empty	Invalid tree
Untuned A	36.01	42.41	41	603	59	104
Cloud A	48.81	54.99	130	480	57	161
Typed A	74.51	82.43	138	655	73	51
Typed B	73.21	81.13	138	645	73	60

The cloud row additionally contains 25 API errors, 57 unparseable outputs, and 10 evaluation failures. The typed-B row contains one evaluation failure. Outcome categories are mutually exclusive within each stored evaluation file, but an outcome such as *answered* can be correct or incorrect.

Table 10: Strict accuracy by depth and declared shape exposure.

Slice	<i>n</i>	Untuned A	Cloud A	Typed A	Typed B
L1	279	50.54	61.29	92.11	90.32
L2	339	27.14	46.90	61.06	59.29
L3	304	32.57	39.47	73.36	73.03
Seen shape	431	46.64	61.48	74.01	72.39
Declared unseen shape	491	26.68	37.68	74.95	73.93

Table 11: Correct cases out of 20 by category in the frozen full-runtime snapshot.

Category	Untuned local	Cloud teacher	Full local
Ambiguous abstention	19	19	19
No-result abstention	20	20	20
Unsupported abstention	19	19	19
Boolean	14	13	14
Bridge join	2	0	14
Categorical	8	11	13
Comparison	7	11	17
Count	17	20	20
Factoid	17	16	15
Minimum/maximum	15	16	20
Set	11	13	15
Sum	20	20	20
Top- k	14	14	18
Overall	183	192	224

Runtime verification and oracle match are separate labels (Table 1), and where both exist for the same item the informative diagnostic is their 2×2 table, which exposes accepted errors and rejected correct plans that a single accuracy figure hides. The stored artifacts do not fill every cell for every experiment, and the missing cells cannot be recovered from final accuracy, so we report the cross-tabulation only for runs whose trace provenance is complete.

Depth and family are separate axes, and neither alone explains difficulty: an L3 set difference can be structurally regular and well covered by supervision, while an L2 grouped concentration query needs a rarer operator and a harder return type. The diagnostic these tables support is a family-by-depth matrix with cell counts printed beside percentages, not a single curve labelled “accuracy versus complexity”.

The repository also contains a 2,285-item final-test summary for several systems. We do not use it as a main result because its summary lacks category/configuration metadata and its adapters are not portable from the current machine-specific output store. Preserving an attractive number without a complete protocol would weaken rather than strengthen reproducibility.

E Trace and Output Examples

E.1 Direct compositional success and base failure

For PACS item PACS::F6:L3:f6_other_buyers_via_suppliers:0011#a, the question asks which buyer-name values are connected through the lead suppliers of an anchor buyer, excluding the exact anchor string. The untuned model emits the following compressed computation:

```
VALUES contract_node_id [supplier_name = anchor]
  DIFFERENCE
VALUES contract_node_id [buyer_name = anchor]
```

It reverses the role in the first filter and projects contract IDs, yielding the empty set. The typed-program model emits:

```
S = VALUES supplier_name [buyer_name = anchor]
B = VALUES buyer_name [supplier_name IN_EXPR S]
A = B DIFFERENCE VALUES buyer_name [buyer_name = anchor]
```

This item ran under the direct typed-program protocol, so the deployment understander, planner, and release checks were not involved: it evidences semantic parsing and deterministic set execution and nothing about the runtime. The deterministic evaluator returns the 17 stored strings in the oracle. Representative values are “Ards and North Down Borough Council”, “Leeds City Council”, and “NI Water and its subsidiaries”. The output also contains organisation-name variants; it must be captioned as 17 stored buyer-name values, not 17 unique organisations.

E.2 Unsupported sibling

A separate F6–L3 control asks, “What is the average number of buyers per supplier in this scope?” Average is outside the closed algebra. The specialised model returns an explicit abstention object with the reason that the operation is not expressible. This is an independent test item, not a failure branch of the answerable bridge question.

E.3 Provenance of the introduction figure

Figure 1 is drawn from a single question, *How many contract notices were published by buyers who have awarded a contract to Edmundson Electrical Limited?*, taken from the balanced comparison set with its template preamble removed. Every panel was re-run under current code against the same graph snapshot, and the full bundle, including retrieved records, emitted programs, and the runtime trace, is stored at `data/qa/figures/fig1/fig1_trace.json` and regenerated by `scripts/paper_figures/fig1_trace.py`.

Computing the denotation directly over U_G gives 23 records naming the supplier, 16 distinct buyer values, and 4,257 notices published by those buyers, which reproduces the frozen oracle. The retrieval panel uses TF-IDF over the record projection at $k = 40$ with field boosting and an untuned Qwen3-8B reader; retrieval recovered all 23 first-hop records and the reader still answered 14. The unsupervised-program panel is the PACS single-decode protocol on the untuned model: its tree passes static validation with root type **Number** and executes to 23, because it filters on the supplier anchor and counts the source records instead of bridging. The typed-program model emits the full bridge and executes to 4,257.

Two limits of the figure should be read with it. The runtime panel was produced by the deterministic rule-based planner: its stored trace records `llm_role: planner_only`, `planner_version: rule_based_planner_v1`, and the stage sequence `planner`, `pre_execution_review`, `grounding`, `retrieval`, `preflight_verifier`, `kg_executor`, `runtime_verifier`, `evidence_verdict`, `answer_sanity`, `reflector`, `answer_card`. The learned understanding and planning stages shown in the figure therefore depict the runtime’s architecture and were not exercised for this item, because the Step-1 and Step-2 adapters are not recoverable from the machine-specific artifact store (Section 6). The evidence bundle also returns up to 200 identifiers rather than one per counted record, so the displayed identifiers are a sample.